Contents

1 Functions **2**
1.1 Calling/invoking a function 2

2 Defining a function (`def`) and its return value (`return`) **2**
2.1 Scope - local and global variables 3
2.2 Composability 4

3 More on `return` **4**
3.1 No `return` statement 5
3.2 Multiple `return` statements 5
3.3 Returning multiple values 6

1 Functions

In the previous labs, we've used many different functions that were written by someone else. For example, `print`, `input`, `math.sin`, and `random.random`, all of which perform some specific set of instructions.

In math, we have functions like $f(x) = \sin x$, which is a function of one variable. It takes in one input, x, and has one output, $\sin x$. We can also have functions which take in more than one input, for example, $f(x, y) = 2xy$. Functions that take in one input and output two things are also possible, for example $f(x) = (x^2, 2x + 1)$.

In Python, or more generally programming, we can define our own functions. We can have as many or as few inputs as we want, and as many or as few outputs as we want. The table below lists a few examples. `print` is somewhat special, being able to take 0 or (infinitely) many inputs.

Function	Inputs	Outputs	Description
`print`	0 or many	returns None	Displays what is passed in
`input`	0 or 1	1	Waits for and returns the user's input
`math.sin`	1	1	Calculates the sine of the input
`random.random`	0	1	Returns a random value in the interval $[0, 1)$

1.1 Calling/invoking a function

To call or invoke a function is to use the function. For example, when we want to display "Hello" to the user, we *call* `print` with the argument `"Hello"`. Specifically, the parentheses `()` after the name of the function indicates we *call* the function to run.

The arguments of the function go within the parentheses.

2 Defining a function (`def`) and its return value (`return`)

There are two important keywords related to functions in Python: `def`, short for 'define', and **`return`**.

It will be easier to start with an example. Say that we want to count the number of vowels in a string repeatedly. Without functions, you may have done it like so:

```
user_in = input("enter a string: ")

vowels = 0
for char in user_in.lower():
  if char in "aeiou":
    vowels += 1
print("there are", vowels, "vowels.")
```

If you wish to repeat this process of counting vowels, you would have to repeat the same lines every time. Instead, we will *abstract* this process of counting vowels into a function:

```
def count_vowels(my_string):                # (1)
  vowels = 0
  for char in my_string.lower():
    if char in "aeiou":
      vowels += 1
  return vowels                             # (2)

user_in = input("enter a string: ")
num_vowels = count_vowels(user_in)          # (3)
print("there are", num_vowels, "vowels.")
```

You can see there are several lines of code which are found in both. For now, observe two important lines: (1), where `def` is used, and (3), where we *call* the function.

In (1), we use `def` to create a function, named `count_vowels`, of *exactly* one input. The intention of the function is to count the number of vowels in the variable `my_string`, which it receives as input. The *body* of the function is all the *indented code* below the `def` line.

After the end of the **`for`** loop in the body of the function, in (2), **`return`** `vowels` is used to set the *return value* of `count_vowels`. The variable `vowels` acts as the counter for the number of vowels in `my_string` - so with `'abcde'`, `vowels` is 2. So `count_vowels` will have a return value of 2 with the input `'abcde'`.

In (3), `count_vowels` is called with `user_in`. When Python encounters this line, it wants to assign to the variable `num_vowels` the value on the right hand side, which is the *return value* of `count_vowels(user_in)`. For Python to know what the return value of `count_vowels(user_in)` is, it has to run the function with the given input. Say that `user_in` is `'abcde'`. Then, Python will run the body of the function, with the variable `my_string` taking on the value `'abcde'`. Outside of this function, `my_string` is not a valid variable.

2.1 Scope - local and global variables

The above section mentions that `my_string` is not a valid variable outside of the function. This behaviour is intended, and keeps Python's environment organised. First, consider the following code block - the body of `count_vowels` remains the same, it is only the code outside the function that has changed.

```
def count_vowels(my_string):
  vowels = 0
  for char in my_string.lower():
    if char in "aeiou":
      vowels += 1
  return vowels

count_vowels("hello")
print(vowels)
```

When this code block is run, a `NameError` occurs: `vowels` is not defined. Even though

`count_vowels` is called, and `vowels` is assigned the value 2 in the process of executing the function, the variable `vowels` no longer exists by the time we try to `print` it. Why?

This is because of *scope*. Whenever a function is called, we enter the *function's scope*, where any variables defined in the body of the function are **discarded** once the function finishes running. For `count_vowels`, this includes the variable `vowels`, the iterating variable **`char`**, and the variable `my_string`, whose value is based on what `count_vowels` was called with.

There is also a *global scope*, where variables are defined **outside** of any function, and are accessible *anywhere* in the code. In the below code, `greeting` is in the global scope, and is used in the `greet` function. This can be convenient, but use it sparingly: a function which depends on global variables is harder to reuse and to reason about, since its behaviour now depends on the state of the rest of the program. As a rule of thumb, anything a function needs should be passed in as an input, and anything it produces should be returned as output.

```
greeting = "hello"                    # defined in the global scope

def greet(name):
  return greeting + ", " + name       # uses global variable `greeting`

print(greet("world"))                 # "hello, world"
```

2.2 Composability

Apart from reducing the need to copy lines of code, abstracting the action of counting vowels in a string into a function allows us to use `count_vowels` in different contexts. It can even be used in other functions, allowing further composition or abstraction.

```
# Count vowels in multiple words
words = ["strong", "green", "omnibus"]
n_vowels = [count_vowels(word) for word in words]
print(n_vowels)

# Usage in another function...
def vowel_proportion(word):
  if len(word) == 0: return 0.0
  return count_vowels(word) / len(word)
print(vowel_proportion("computational"))

# ... which is used in another different context.
vowelly_words = [word for word in words if vowel_proportion(word) >= 0.4]
print(vowelly_words)
```

3 More on `return`

The usage of the **`return`** keyword is important.

3.1 No `return` statement

Without **`return`**, a function will always return the special value `None`. For example, in the above example, if we omit **`return`** `vowels` from the function, it still performs the action of counting vowels - `vowels` will still be 2 - but the return value is no longer informative. Instead, we will see that there are `None` vowels.

```
def count_vowels(my_string):
  vowels = 0
  for char in my_string.lower():
    if char in "aeiou":
      vowels += 1
  # return vowels

user_in = input("enter a string: ")
num_vowels = count_vowels(user_in)
print("there are", num_vowels, "vowels.")
```

One solution you may consider is to replace the **`return`** statement in the function body with the `print` function. This, however, prevents `count_vowels` from being used in different contexts, since it no longer has a proper return value. For example, you may not want to print something every time you run `count_vowels`.

3.2 Multiple `return` statements

A function can also have more than one **`return`** statement. The return value of the function will be the **first** return statement that Python encounters in the process of running the code. Importantly, the function ends at the first return statement encountered.

In the below example, Python will print, and then return 1. All subsequent lines in the function after **`return`** 1 will never be able to be executed.

```
def f():
  print("returning 1!")
  return 1
  print("this will never be printed!")
  return 2

f()
```

It is more useful when there are branching conditional statements, when only one **`return`** statement will ever run:

```
def is_even(n):
  if n % 2 == 0:
    return True
  else:
    return False
```

On the other hand, consider when there are **`return`** statements which may never be encountered.

```
def is_even(n):
  if n % 2 == 0:
    return True
  # what is returned if n is odd?
```

3.3 Returning multiple values

Functions can also return more than one value. Separate the items to return with commas, and they will be returned as a tuple, from which the entries can be indexed.

If a function returns more than 1 value, the return value can be immediately deconstructed into its component parts.

```
import math

def polar_coord(r, theta):
  return r * math.cos(theta), r * math.sin(theta)

coord = polar_coord(1, 0.78539816)    # r = 1, theta = pi/4
print(coord)
print(coord[0], coord[1])

# Deconstruct the return value
x, y = polar_coord(1, 0.78539816)     # r = 1, theta = pi/4
print(x, y)
```